



RELATÓRIO TÉCNICO

Atividade 7: Timer

ALUNOS

Aluno(a): Tiago Navarro Bettanin

R.A: 2477866

Aluno(a): João Antonio Sitta Martins

R.A: 2553058

1 Introdução

Nesta atividade, o objetivo foi desenvolver, em VHDL, um **timer regressivo** capaz de contar o tempo em segundos até chegar a zero. Ao fim da contagem, o circuito deve acender um LED indicador e apresentar o valor atual do tempo em quatro displays de sete segmentos (SSDs). O projeto também inclui um botão de **reset assíncrono**, um botão de **pausa/desabilitação** da contagem e diferentes opções de tempo inicial selecionadas por meio de uma entrada de seleção.

Os principais conceitos explorados estão relacionados ao uso de **elementos de biblioteca em VHDL**, conteúdo abordado na Aula 11. O projeto foi organizado com um *package*, utilizado para agrupar funções reutilizáveis, e uma entidade principal, responsável pela lógica sequencial do timer. O uso de *package* facilita a organização do código, pois separa funções auxiliares, como a conversão para BCD e a decodificação para display de sete segmentos, da arquitetura principal do circuito.

Outro conceito importante é o uso de **function**. Em VHDL, uma função executa comandos sequenciais internamente e retorna apenas um valor. Nesta atividade, esse recurso foi usado para transformar o valor inteiro da contagem em quatro dígitos BCD e, posteriormente, converter cada dígito BCD no padrão necessário para acionar um SSD. Dessa forma, o código principal fica mais simples e a lógica de conversão pode ser reutilizada em outros projetos.

2 Desenvolvimento

2.1 Descrição geral do circuito

A solução foi dividida em dois arquivos. O arquivo `timer_pkg.vhd` contém o *package* `timer_pkg`, no qual foram implementadas duas funções: `int_to_bcd`, responsável por converter o valor inteiro do tempo para quatro dígitos BCD, e `bcd_to_7seg`, responsável por transformar cada dígito BCD no código de sete segmentos correspondente. O arquivo `atividade7.vhd` contém a entidade principal do timer.

A entidade `atividade7` possui o genérico `CLK_FREQ`, definido inicialmente como 50000000, representando uma frequência de clock de 50 MHz. Esse valor é usado para gerar um pulso de um ciclo de clock a cada segundo, chamado de `tick_1hz`. Sempre que esse pulso ocorre e a contagem está habilitada, o valor de `seconds` é decrementado.

A entrada `sel` possui dois bits e define o tempo inicial da contagem. Foram implementadas quatro opções de tempo, conforme apresentado na Tabela 1. Com isso, o requisito de possuir ao menos duas opções diferentes de contagem foi atendido.

Tabela 1: Tempos iniciais selecionados pela entrada `sel`.

Valor de <code>sel</code>	Tempo inicial carregado
00	10 segundos
01	30 segundos
10	100 segundos
11	1000 segundos

O sinal `reset_n` é ativo em nível baixo e executa o reset de forma assíncrona. Quando esse botão é pressionado, o contador recebe o valor definido por `preset`. A entrada `pause_n` controla a habilitação da contagem: quando está em nível lógico alto, o timer decrementa normalmente; quando está em nível lógico baixo, a contagem fica pausada. Ao chegar em zero, o sinal `led_done` recebe nível lógico alto, indicando o fim da contagem.

2.2 Conversão do valor para os displays

Para apresentar o tempo nos quatro SSDs, o valor inteiro armazenado em `seconds` precisa ser separado em unidades, dezenas, centenas e milhares. Essa separação é feita pela função `int_to_bcd`. Em vez de implementar o algoritmo *double dabble*, a função utiliza operações aritméticas de divisão e resto para obter diretamente cada dígito decimal: `value mod 10` para unidades, `(value / 10) mod 10` para dezenas, `(value / 100) mod 10` para centenas e `(value / 1000) mod 10` para milhares.

O resultado é um vetor de 16 bits, no qual cada grupo de 4 bits representa um dígito BCD. Em seguida, cada grupo é passado para a função `bcd_to_7seg`, que usa uma estrutura `case` para retornar o padrão de segmentos correspondente aos números de 0 a 9. Por fim, esses padrões são ligados às saídas `hex0`, `hex1`, `hex2` e `hex3`.

2.3 Código VHDL do package

O Código 1 apresenta o arquivo `timer_pkg.vhd`. Esse arquivo concentra as funções auxiliares do projeto, deixando a entidade principal responsável apenas pelo controle da contagem, reset, pausa e seleção de tempo.

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 package timer_pkg is
6
7     function int_to_bcd(value : integer) return std_logic_vector;
8
9     function bcd_to_7seg(bcd : std_logic_vector(3 downto 0)) return
        std_logic_vector;
10

```

```

11 end package timer_pkg;
12
13
14 package body timer_pkg is
15
16     function int_to_bcd(value : integer) return std_logic_vector is
17         variable bcd : std_logic_vector(15 downto 0);
18     begin
19         bcd(3 downto 0) := std_logic_vector(to_unsigned(value mod 10, 4)
20         );
21         bcd(7 downto 4) := std_logic_vector(to_unsigned((value / 10) mod
22         10, 4));
23         bcd(11 downto 8) := std_logic_vector(to_unsigned((value / 100)
24         mod 10, 4));
25         bcd(15 downto 12) := std_logic_vector(to_unsigned((value / 1000)
26         mod 10, 4));
27         return bcd;
28     end function int_to_bcd;
29
30     function bcd_to_7seg(bcd : std_logic_vector(3 downto 0)) return
31     std_logic_vector is
32     begin
33         case bcd is
34             when "0000" => return "0000001"; -- 0
35             when "0001" => return "1001111"; -- 1
36             when "0010" => return "0010010"; -- 2
37             when "0011" => return "0000110"; -- 3
38             when "0100" => return "1001100"; -- 4
39             when "0101" => return "0100100"; -- 5
40             when "0110" => return "0100000"; -- 6
41             when "0111" => return "0001111"; -- 7
42             when "1000" => return "0000000"; -- 8
43             when "1001" => return "0000100"; -- 9
44             when others => return "0110000";
45         end case;
46     end function bcd_to_7seg;
47
48 end package body timer_pkg;

```

Listing 1: Arquivo timer_pkg.vhd – package com as funções de conversão para BCD e sete segmentos.

A função `int_to_bcd` retorna um vetor `std_logic_vector(15 downto 0)`, suficiente para representar quatro dígitos BCD. A função `bcd_to_7seg` recebe um único dígito BCD de 4 bits e retorna um vetor de 7 bits, no qual cada bit corresponde a um segmento do display. Como cada função retorna apenas um valor, sua utilização está de acordo com a estrutura de *function* apresentada na Aula 11.

2.4 Código VHDL da entidade principal

O Código 2 apresenta o arquivo `atividade7.vhd`. Nele, o *package* é incluído por meio da linha `use work.timer_pkg.all;`, permitindo que as funções `int_to_bcd` e `bcd_to_7seg` sejam chamadas diretamente dentro da arquitetura.

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4 use work.timer_pkg.all;
5

```

```

6  entity atividade7 is
7      generic (
8          CLK_FREQ : integer := 50000000
9      );
10     port (
11         clk      : in  std_logic;
12         reset_n   : in  std_logic;
13         pause_n   : in  std_logic;
14         sel       : in  std_logic_vector(1 downto 0);
15         led_done  : out std_logic;
16         hex0      : out std_logic_vector(6 downto 0);
17         hex1      : out std_logic_vector(6 downto 0);
18         hex2      : out std_logic_vector(6 downto 0);
19         hex3      : out std_logic_vector(6 downto 0)
20     );
21 end entity atividade7;
22
23
24 architecture atividade7_arch of atividade7 is
25     signal seconds : integer range 0 to 9999;
26     signal preset  : integer range 0 to 9999;
27     signal tick_1hz : std_logic;
28     signal bcd_val  : std_logic_vector(15 downto 0);
29 begin
30
31     process(sel)
32     begin
33         case sel is
34             when "00" => preset <= 10;
35             when "01" => preset <= 30;
36             when "10" => preset <= 100;
37             when others => preset <= 1000;
38         end case;
39     end process;
40
41     process(clk, reset_n)
42         variable cnt : integer range 0 to CLK_FREQ - 1;
43     begin
44         if reset_n = '0' then
45             cnt := 0;
46             tick_1hz <= '0';
47         elsif rising_edge(clk) then
48             if cnt = CLK_FREQ - 1 then
49                 cnt := 0;
50                 tick_1hz <= '1';
51             else
52                 cnt := cnt + 1;
53                 tick_1hz <= '0';
54             end if;
55         end if;
56     end process;
57
58     process(clk, reset_n)
59     begin
60         if reset_n = '0' then
61             seconds <= preset;
62         elsif rising_edge(clk) then
63             if pause_n = '1' and tick_1hz = '1' and seconds > 0 then
64                 seconds <= seconds - 1;
65             end if;

```

```

66     end if;
67 end process;
68
69 led_done <= '1' when seconds = 0 else '0';
70
71 bcd_val <= int_to_bcd(seconds);
72
73 hex0 <= bcd_to_7seg(bcd_val(3 downto 0));
74 hex1 <= bcd_to_7seg(bcd_val(7 downto 4));
75 hex2 <= bcd_to_7seg(bcd_val(11 downto 8));
76 hex3 <= bcd_to_7seg(bcd_val(15 downto 12));
77
78 end architecture atividade7_arch;

```

Listing 2: Arquivo atividade7.vhd – entidade principal do timer regressivo.

A arquitetura pode ser entendida em quatro blocos principais. O primeiro bloco define o valor de **preset** conforme a entrada **sel**. O segundo bloco divide o clock de entrada para gerar o sinal **tick_1hz**. O terceiro bloco implementa o contador regressivo, com reset assíncrono e pausa. O quarto bloco converte o valor de **seconds** para BCD e depois para os sinais dos quatro displays de sete segmentos.

2.5 Funcionamento esperado

Durante o reset, o contador recebe o valor inicial selecionado em **sel**. Após a liberação do reset, se **pause_n** estiver em nível alto, o contador decrementa uma unidade por segundo. Se **pause_n** for colocado em nível baixo, o valor de **seconds** permanece armazenado, fazendo com que a contagem fique pausada. Quando **seconds** chega a zero, o decremento para, os displays passam a apresentar 0000 e a saída **led_done** é acionada.

Tabela 2: Resumo do comportamento dos sinais de controle.

Sinal	Condição	Comportamento
reset_n	0	Carrega o valor de preset e reinicia a divisão de clock
pause_n	1	Habilita a contagem regressiva
pause_n	0	Pausa/desabilita a contagem
seconds	0	Aciona led_done

2.6 Diagrama RTL

A síntese no Quartus Prime gera o diagrama RTL do circuito, permitindo visualizar a estrutura de hardware produzida a partir do código VHDL. Espera-se observar blocos associados ao divisor de clock, ao registrador da contagem, à lógica de seleção do **preset**, à comparação com zero, à conversão para BCD e aos decodificadores dos quatro displays de sete segmentos.

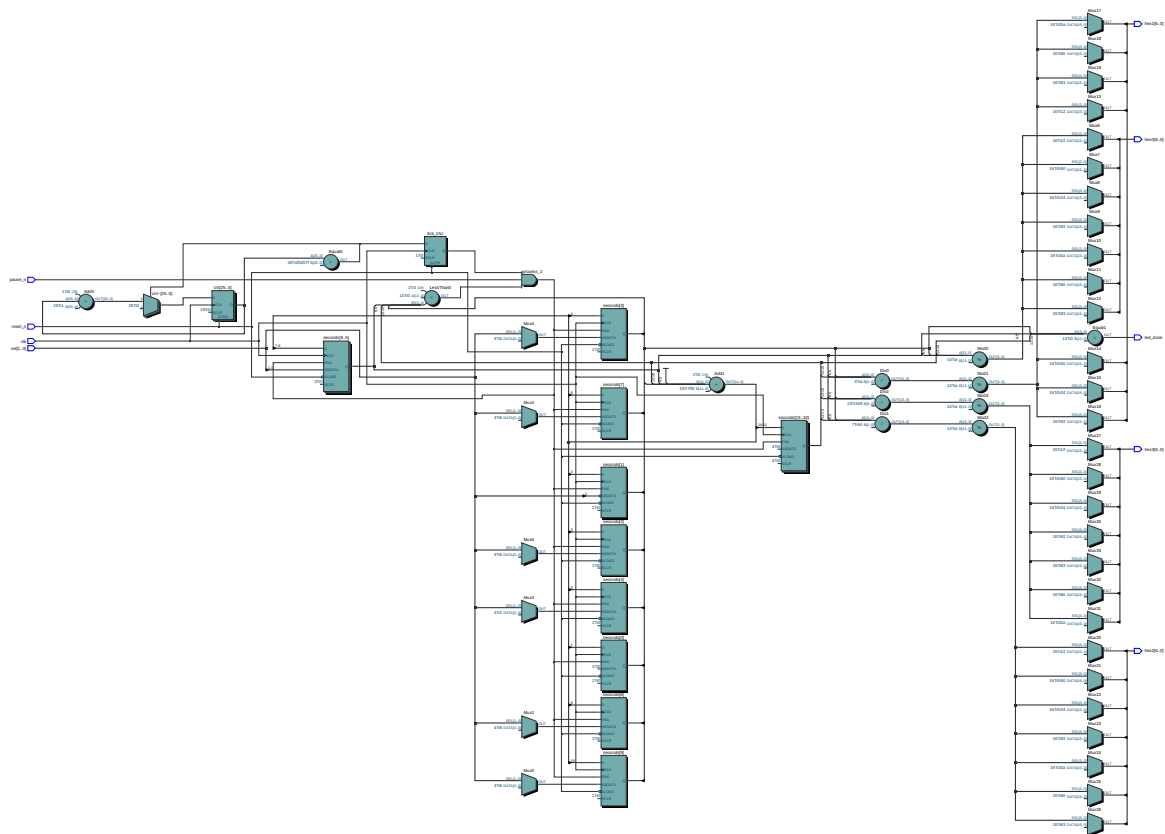


Figura 1: Diagrama RTL do timer regressivo, gerado pelo Quartus Prime.

3 Conclusão

A atividade permitiu aplicar os conceitos de organização de código em VHDL por meio de *package* e *function*, além de reforçar a implementação de circuitos sequenciais utilizando processos sensíveis ao clock e ao reset. O uso do `timer_pkg` tornou o projeto mais organizado, separando as funções de conversão para BCD e sete segmentos da lógica principal do timer.

O circuito desenvolvido atende aos requisitos propostos para a Atividade 7: realiza contagem regressiva até zero, aciona um LED ao final da contagem, apresenta o tempo em quatro SSDs, possui reset assíncrono, permite pausar/desabilitar a contagem e disponibiliza diferentes opções de tempo inicial. Além disso, a utilização do genérico `CLK_FREQ` facilita a adaptação do projeto para placas com frequências de clock diferentes, mantendo a geração do pulso de um segundo como parte parametrizável do sistema.

Com isso, o projeto demonstra uma aplicação prática dos elementos de biblioteca estudados na Aula 11, principalmente *package* e *function*, dentro de um sistema digital funcional e modular.

Referências

- [1] Atividade 7: Timer. Documento de enunciado fornecido pelo professor, disciplina de Lógica Reconfigurável, UTFPR Campus Apucarana, 2025/2.
- [2] Marcelo de Oliveira. *Aula 11 – Elementos de biblioteca: package, component, function e procedure*. Disciplina de Lógica Reconfigurável, UTFPR Campus Apucarana.